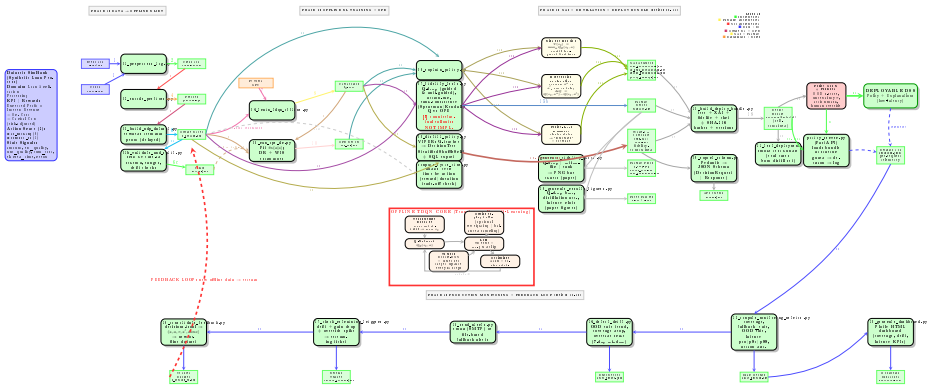

Figure 1: **End-to-end implementation architecture (actual state) of xPPM-TDQN: Explainable Prescriptive Process Monitoring with Transformer Deep Q-Learning.** All coloured boxes are fully implemented. **Phase 1** preprocesses SimBank event logs (2-action space: `do_nothing` vs `contact_HQ`), encodes prefixes, builds the MDP dataset with terminal-profit reward, and produces stratified train/val/test splits. **Phase 2** trains an offline Double-DQN with a Transformer encoder (d_model=128, 4 heads, 3 layers) and evaluates the learned policy off-policy with doubly robust + WIS estimators. **Phase 3** generates three explanation types (risk via Integrated Gradients, ΔQ contrastive, policy-level clustering), runs fidelity tests (Q-drop guided & anti-guided, action-flip, rank-consistency Spearman/Kendall), distills the Q-teacher to a decision tree via VIPER (StratifiedShuffleSplit + SQL rule export), exports the API JSON Schema (script 09), bundles all artifacts with SHA-256 content hashes (script 10), and smoke-tests the bundle on real distillation cases (script 11). The FastAPI server loads the bundle at startup and filters every decision through a Policy Guard (OOD z-score, uncertainty threshold, action mask, human override). **Phase 4** closes the production loop: the server logs each decision to `decisions.jsonl`; scripts 13–18 compute daily metrics, detect drift (OOD trend, coverage drop, override spike), dispatch alerts, check retraining triggers, consolidate feedback into a new offline dataset (`D_offline_vN.npz`), and generate a Plotly monitoring dashboard. Research utilities (`generate_fidelity_plots.py`, `19_generate_results_figures.py`, `compute_cycle_time.py`) are shown separately. **Note:** Counterfactual rollouts in fidelity tests are not yet implemented.